FinRelief AI

Debt Relief & Financial Recovery Platform

Project Documentation

Stack: React (Vite + Tailwind) · FastAPI · SQLite · SQLAlchemy · Google Gemini API

Prepared: July 2026

Table of Contents

1. Project Overview

FinRelief AI is an AI-powered web application designed to help borrowers manage their loans, understand their financial health, receive AI-driven settlement predictions, and generate lender-specific negotiation letters. The platform combines deterministic, rule-based financial calculations with generative AI (Google Gemini) to give users clear, actionable guidance for resolving debt.

The application is built as a full-stack system with a React single-page frontend and a FastAPI backend, backed by a SQLite relational database that mirrors an entity-relationship (ER) design covering users, their financial profiles, loans, settlement predictions, negotiation records, and an aggregated AI activity history.

---

2. Technology Stack

Layer	Technology	Purpose
Frontend	React.js (Vite) + Tailwind CSS	Single-page application, fast dev/build tooling, utility-first styling
Backend	FastAPI (Python)	REST API layer, request validation, routing
Database	SQLite + SQLAlchemy ORM	Persistent storage of users, loans, predictions, history
AI Integration	Google Gemini API (gemini-1.5-flash)	Natural-language negotiation strategy & letter generation
Authentication	JWT (python-jose) + bcrypt (passlib)	Stateless session auth with hashed credentials
HTTP Client	Axios	Frontend-to-backend requests with auth interceptor

---

3. System Architecture

The application follows a layered enterprise architecture:

- User Layer — end users interacting through a web browser
- Frontend Layer — React components, pages, and context providers
- API Layer — FastAPI routers exposing REST endpoints (backend/routers/)
- AI & Financial Processing Layer — business logic and AI integration (backend/services/)
- Database Layer — SQLite database accessed via SQLAlchemy models

Requests flow from the browser to React pages, which call the backend through an Axios client with an authentication interceptor. FastAPI routers validate requests using Pydantic schemas and delegate business logic to the services layer, which performs financial calculations, settlement predictions, and AI-assisted letter generation before persisting results to SQLite.

4. Folder Structure

```
finrelief-ai/
├── backend/
│   ├── main.py                 # FastAPI app entrypoint
│   ├── database.py             # SQLAlchemy engine/session
│   ├── models.py               # DB models (ER diagram)
│   ├── schemas.py              # Pydantic request/response schemas
│   ├── auth.py                 # JWT auth + password hashing
│   ├── requirements.txt
│   ├── routers/                # API layer
│   │   ├── auth_router.py
│   │   ├── loans_router.py
│   │   ├── financial_router.py
│   │   ├── settlement_router.py
│   │   ├── negotiation_router.py
│   │   └── history_router.py
│   └── services/               # AI & financial processing layer
│       ├── financial_engine.py
│       ├── settlement_engine.py
│       ├── gemini_service.py
│       └── negotiation_engine.py
└── frontend/
    └── src/
        ├── pages/            # Login, Register, Dashboard, Loans,
        │                     # Settlement, Negotiation, History
        ├── components/       # Navbar, StatCard, ProtectedRoute
        ├── context/          # AuthContext (JWT + user state)
        └── api/client.js     # Axios instance with auth interceptor
```

5. Data Model (ER Diagram Summary)

The relational schema in models.py implements the following entities and relationships:

Entity	Relationship	Notes
Users	1 : 1 → Financial_Profile	Income, expenses, and derived financial metrics
Users	1 : N → Loans	A user can hold multiple loan accounts
Users	1 : N → AI_History	Aggregated log of all AI-driven activity
Loans	1 : N → Settlement_Prediction	Each prediction run is stored for audit/history
Loans	1 : N → AI_Negotiation	Each generated letter/strategy is stored for audit/history

---

6. Core Functional Engines

6.1 Financial Health Engine (rule-based, deterministic)

- EMI Ratio = total monthly EMI ÷ monthly income
- DTI Ratio = (total EMI + other expenses) ÷ monthly income
- Monthly Surplus = income − expenses − EMI
- Stress Level = weighted score from DTI ratio, surplus, and overdue months, classified as Low / Moderate / High / Critical

6.2 Settlement Prediction Engine (rule-based)

Begins from an 80% base settlement offer and applies discounts based on overdue months, stress level, and interest rate to produce a suggested settlement percentage, a risk category, and a predicted payoff amount.

6.3 AI Negotiation Letter Generator

Calls Google Gemini (gemini-1.5-flash) first to produce a tailored negotiation strategy and letter. If no API key is configured, or the Gemini call fails for any reason (network issues, quota limits, etc.), the system automatically falls back to a deterministic, template-based generator — ensuring the feature always works, even offline or during a live demo.

6.4 Authentication

JWT-based authentication with bcrypt-hashed passwords (python-jose + passlib). Tokens are stored client-side in localStorage and attached to every API request via an Axios interceptor.

---

7. API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Create account, returns JWT

Method	Endpoint	Description
POST	/api/auth/login	Login, returns JWT
GET	/api/auth/me	Current user profile
PUT	/api/auth/me/income	Update income/expenses
GET / POST / PUT / DELETE	/api/loans/	Loan CRUD
GET	/api/financial/profile	Financial health metrics
POST	/api/settlement/predict/{loan_id}	Run settlement prediction
GET	/api/settlement/loan/{loan_id}	Prediction history for a loan
POST	/api/negotiation/generate	Generate negotiation strategy + letter
GET	/api/negotiation/loan/{loan_id}	Negotiation history for a loan
GET	/api/history/	Full AI activity history

Full interactive API documentation is auto-generated by FastAPI at /docs (Swagger UI).

---

8. Setup & Run Instructions

8.1 Backend

```
cd backend
python -m venv venv
source venv/bin/activate      # Windows: venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env          # then edit .env
uvicorn main:app --reload --port 8000
```

Set SECRET_KEY (any long random string) and, optionally, GEMINI_API_KEY in .env. If GEMINI_API_KEY is left blank, the app automatically uses the rule-based negotiation fallback. The SQLite database and tables are created automatically on first run.

8.2 Frontend

```
cd frontend
npm install
npm run dev
```

The app runs at http://localhost:5173. Vite proxies /api requests to http://localhost:8000, so the backend must be started first.

---

9. Key Features

- Secure user registration & login (JWT + bcrypt)
- Loan management — add, edit, delete loan accounts
- Financial health dashboard — EMI ratio, DTI ratio, surplus, stress level
- AI-driven settlement prediction with risk categorization
- AI negotiation strategy & letter generator (Gemini-powered, with offline fallback)
- Full AI activity history log for auditability

---

10. Demo Video

A screen-recorded walkthrough (14.07.2026_18.42.39_REC.mp4) demonstrates the complete working application end-to-end:

- User registration and login
- Adding and managing loan accounts
- Financial health dashboard walkthrough
- AI-powered settlement prediction for a loan
- AI negotiation letter generation
- AI history log of past predictions and letters